Laboratorium Task 2

by Paweł Paduch (https://achilles.tu.kielce.pl/portal/author/332bd2d158a24964b2b98a7fc2879842) published 2 years ago, last modified 2 years ago

Bardziej zaawansowane zagadnienia tworzenia i zarządzania zadaniami.

1. **ExecuteSynchronously (0,5 pkt)**
   - **W nowej solucji utworzyć projekt LabTask1.**
   - **Za pomocą** `Task task = Task.Factory.StartNew(() =>{}` **utworzyć pierwszy *task* (na razie pusty)**
   - **Następnie w pętli milion razy wywołać kontynuację taska na zasadzie wykonuję jeden po drugim.**

     ```
     Task t = task.ContinueWith((ant, a) =>
     {}, i
     //,TaskContinuationOptions.ExecuteSynchronously
     );
     task = t;
     ```

   - **Za pomocą** `Stopwatch` **zmierzyć czas inicjalizacji (po pętli tworzącej milion tasków). Uwaga pomiary czasu wykonujemy bez opcji *Debug*. Czyli przez ctrl+F5.**
   - **Poczekać na zakończenie taska *task* i znowu złapać czas.**
   - **Odkomentować** `TaskContinuationOptions.ExecuteSynchronously`
   - **Dokonać nowych pomiarów.**
     Należy wykonać po 5 pomiarów i wyciągnąć średnią.

2. **ExecuteSynchronously z cięższymi zadaniami (0,5 pkt)**
   - **Dodaj funkcję uruchamianą w każdym tasku, która rozgrzeje nieco procesor:**
   - ```
     static void DoPointlessMess()
     {
         double aa = 0;
         for (int i = 0; i < 100; i++)
         {
             aa++;
             aa = (Math.Sin(aa) + Math.Cos(aa));
             aa = aa % Math.Sin(aa);
             aa = Math.Log2(Math.Tanh(aa));
             aa *= Math.Sin(aa) / Math.Cos(aa) == 0 ? 1 : Math.Cos(aa);
         }
     }
     ```

   - **Wykonaj powyższe pomiary włączając i wyłączając** `TaskContinuationOptions.ExecuteSynchronously` **. Czy coś się zmieniło?**

3. **ExecuteSynchronously z przekazaniem wartości. (0,5 pkt.)**
   - **ContinueWith używa się zwykle do przekazywania danych z Taska do Taska.**
     Przerobić funkcję `DoPointlessMess` **tak, by zwracała wynik obliczeń a zmienna *aa* była przekazywana z zewnątrz**
   - **Przerobić wywołanie** `Task.Factory.StartNew` **tak, by uwzględniało przekazywanie danych z i do funkcji** `DoPointlessMess` **.**
   - **Wykonać testy jak w poprzednim ćwiczeniu.**

4. **Ciąg Fibonacciego (1 pkt.)**
     ```

- Załóż nowy projekt. Skopiuj i przerób powyższe projekt tak by program liczył ciąg Fibonacciego.
- W celu obliczeń naprawdę dużych liczb użyj typu *BigInt*.
- Przekazywanie pary liczb można wykonać za pomocą krotki (*Tupple<BigInt, BigInt>*)
- Można przetestować działanie np tworząc 200tys tasków.

```
static Tuple<BigInteger, BigInteger> BigFibonacci(BigInteger n2, BigInteger n1, int
{
    // Console.WriteLine($"{n2}");
    return Tuple.Create(n1,n1+n2);
}
```

- Z ciekawości można podejrzeć generowane liczby, lub sam wynik końcowy (uwaga finalna liczba zajmuje kilka ekranów :))
- Dla kolejnych ciekawskich, porównać czas wykonania z rozwiązaniem jednowątkowym:

```
BigInteger n2 = 0;
BigInteger n1 = 1;
for (int i = 0; i <= 200000; i++)
{
    Tuple<BigInteger, BigInteger> result = BigFibonacci(n2, n1, n);
    n1 = result.Item2;
    n2 = result.Item1;
}
```

5. **Liczby pierwsze (1,5 pkt)**
   - Załóż nowy projekt bazując na poprzednim.
   - Użyj poniżej funkcji do wyliczania liczb pierwszych w zakresie

```
int countFromTotal = 0;
int countToTotal = 100000000;
```

```
public static bool IsPrime(int liczba)
{
  if (liczba <= 1) return false; // Liczby mniejsze lub równe 1 nie są pierwsze
  if (liczba <= 3) return true; // 2 i 3 są liczbami pierwszymi

  // Sprawdzamy podzielność przez 2 i 3
  if (liczba % 2 == 0 || liczba % 3 == 0) return false;

  // Sprawdzamy podzielność przez liczby postaci 6k ± 1, gdzie k = 1, 2, 3, ...
  for (int i = 5; i * i <= liczba; i += 6)
  {
    if (liczba % i == 0 || liczba % (i + 2) == 0)
    return false;
  }
  return true;
}
```

   - Liczby należy zapisać do listy `List<int> singleThreadPrimes = new List<int>();`
   - Należy zmierzyć czas wykonania dla pojedynczego wątka.
   - Następnie dorób do bieżącego projektu wyliczanie liczb pierwszych bazujące na taskach.
     - Każdy task otrzymuje liczbę początkową i końcową zakresu, który ma sprawdzić.
     - Każdy task zwraca listę liczb pierwszych
     - Liczba tasków do uruchomienia powinna być odczytana za pomocą `Environment.ProcessorCount`

- Za pomocą `Task.Factory.ContinueWhenAll` utworzyć finalny task zbierający dane od pozostałych tasków.
- Task ten powinien połączyć zebrane listy *intów* w jedną listę i ją zwrócić.
- Zmierzyć czas znalezienia wszystkich liczb z podanego zakresu dla wersji wielowątkowej.

6. **Utworzyć klasę statyczną *PrimesGenerator (1 pkt)***
   - Utworzyć asynchroniczną metodę zwracającą zadanie zwracające listę int
     ```
     public static async Task<List<int>> FindPrimesAsync(int start, int end)
     ```
   - W metodzie tej podobnie jak poprzednim zadaniu powinniśmy rozdzielić główne zadanie na mniejsze podzadania, poczekać na wyniki i zwrócić zadanie zcalające. Można posłużyć się wyrażeniem LINQ.
     ```
     List<int>[] results = await Task.WhenAll(tasks);
     return results.SelectMany(primes => primes).ToList();
     ```
   - Użyć w programie. Zmierzyć czas wykonania.